Agent 时代，为什么“大家一起改同一个软件”不够了？

“通过 ASKS 智能知识系统看 AI Agent”系列第 6 篇

ASKS（Agent-Driven Scientific Knowledge System）是我们的智能知识系统。

项目代码：https://github.com/ranshiju/Ran-ASKS

本文通过与 ASKS 系统交互完成。



共享底座支持不同领域系统，真实使用中验证出的通用改进再回到底座

1991 年，Linux kernel 还只是一个很小的项目。今天，它早已不是“大家一起编辑一个文件”的规模。

Linux kernel 的开发文档显示，内核被分成许多子系统，各有维护者。一个补丁通常先经过对应社区评审，再进入子系统代码树，最后才可能进入主线。与此同时，Ubuntu 与 Fedora 等发行版并没有合并成同一个操作系统：它们围绕共同的内核和大量上游软件，做出不同的软件选择、发布节奏和用户体验。

更有意思的是，变化并不只从上游流向下游。Fedora 把“upstream first”视为核心原则：如果一个改进具有共同价值，尽量把它送回原项目。这样，更多人可以受益，Fedora 自己也不必长期背着一套越来越难维护的专用补丁。

这套生态已经证明：合作不等于做出同一个最终产品。人们可以共享底层成果，让不同产品独立生长，再把有共同价值的改进送回上游。

到了 Agent 时代，这个思路变得更加重要，也更加困难。因为一个 Agent 系统的差异，不只藏在代码里，还会进入它怎样理解任务、依据什么知识行动、能够接触哪些数据，以及用什么标准判断“做得好不好”。

为什么一个团队很难包办所有事情？

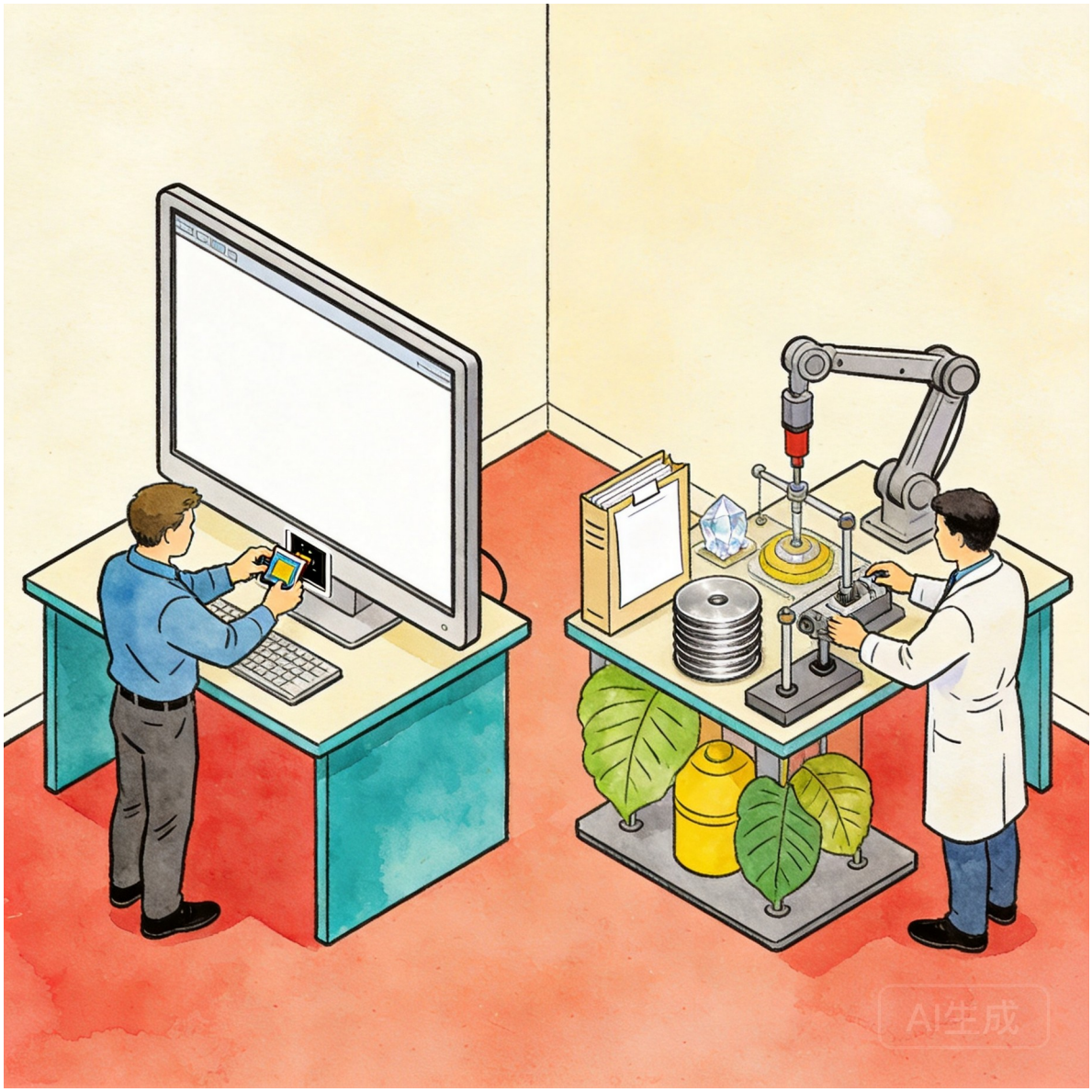
设想一个材料研究团队，想基于 ASKS 建设自己的智能知识系统。

它需要读取材料论文、连接专业数据库、识别实验条件和物理量，还要回答真正的研究问题。通用系统的开发者可以提供来源追踪、受控写入、权限、事务、校验和发布机制，却不可能凭空知道材料学家每天怎样工作、哪些概念最容易混淆、什么样的回答在专业上才算合格。

反过来，领域团队拥有专业知识和真实场景，却未必值得从头再造账户权限、版本升级、失败回滚、来源记录和发布检查。每个领域都重新造一遍底座，既浪费，也会重复踩同样的坑。

真实用户同样不可缺少。一个系统在演示里能完成任务，不等于在日常工作里稳定有用。只有放进真实场景，人们才会发现：哪些能力确实通用，哪些只是某个领域的习惯，哪些错误会带来真正的后果。

因此，Agent 开发天然需要三种力量同时在场：通用工程提供可靠底座，领域团队提供知识和工作方法，真实使用与评测提供证据。协作的目的，不是把所有人拉进同一个代码库，而是让这三种力量能够互相连接。



浏览器扩展围绕清楚接口增加功能，Agent 下游还会改变任务、知识、工具和评测

与浏览器扩展相比，Agent 协作改变了什么？

浏览器扩展是一个容易理解的传统例子。按照 Mozilla 的 WebExtensions 说明，扩展可以通过一组额外的接口为浏览器增加广告过滤、翻译、密码管理等功能，而且许多接口能在多个主流浏览器之间复用。

这里的分工相对清楚：浏览器决定主要运行方式，扩展围绕预先定义的接口增加功能。两者当然也会遇到权限、兼容和安全问题，但开发者通常知道“扩展插在哪里”。

材料研究 Agent 的变化可能深入得多。它也许要重新设计：

- 什么算一个任务，任务应该怎样拆分；
- 哪些论文和数据库可以作为依据；
- Agent 使用哪些提示词、工具和工作流程；
- 私有实验数据能否送到远程模型；
- 什么测试只能检查格式，什么评测才能判断专业质量。

这些变化共同决定系统最后会怎样行动。也就是说，领域团队不一定只是在共同软件上“加一个按钮”，它可能正在长出一个行为不同、责任独立的下游系统。

插件、发行版、Fork 和上下游协作早已存在，Agent 并没有凭空发明它们。真正的新变化是：协作对象从代码和接口，进一步扩展到任务约定、知识来源、数据边界、Agent 行为和质量证据。代码仍然重要，却已经不能单独说明两个 Agent 系统是否真的“兼容”或“同样好用”。

新协作首先要回答四个问题

材料团队开始改造系统后，很快会碰到四个问题。

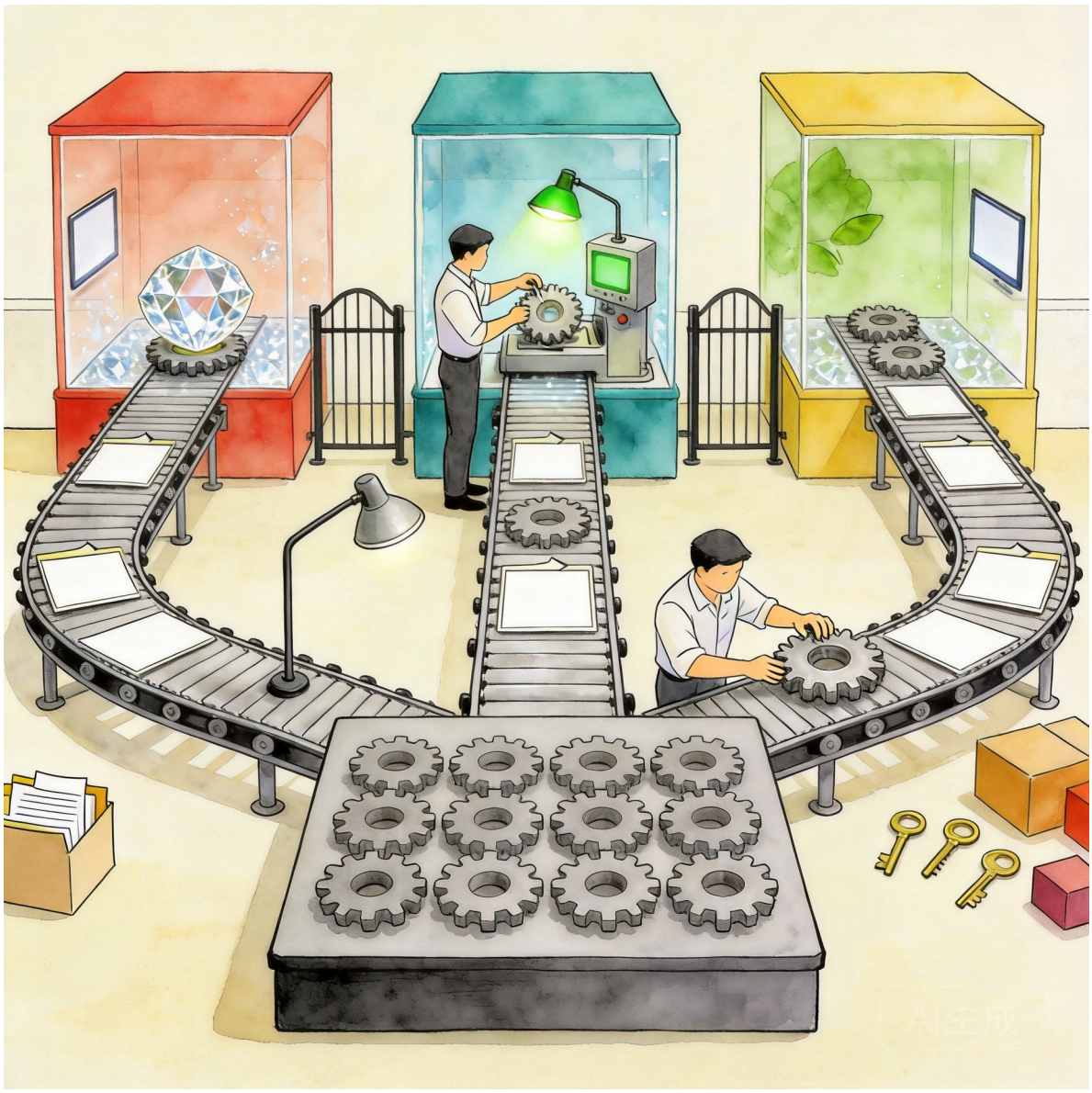
第一，什么应该留在共同底座？来源追踪和失败回滚可能适用于许多领域；某种材料晶体结构的识别规则，通常只属于材料系统。

第二，怎样说明双方仍然兼容？“代码来自同一个项目”只能说明出身，不能证明当前版本仍支持同样的交接格式和检查方法。

第三，出了问题谁负责？材料数据库的许可、领域回答的质量、系统升级和用户支持，都需要明确的维护者。

第四，真实场景里发现的改进怎样惠及其他人？如果材料团队修复了一个通用事务缺陷，这个补丁不应永远困在材料系统里；如果它只是材料领域的专用提示词，也不必硬塞进所有人的底座。

这四个问题指向同一种协作需求：共同部分要足够稳定，领域部分要有独立生长空间，真正通用的进步还要有路可回。



共享底座—独立下游—证据回流，让共同工程与领域创新形成循环

一种协作模式：共享底座—独立下游—证据回流

可以把这种模式概括成三步。

第一步是共享底座。共同维护跨领域都需要的能力，例如来源追踪、受控写入、权限、事务、结构校验和发布安全。共享的还不只是代码，也包括任务怎样交接、结果怎样检查等约定。

第二步是独立下游。材料团队拥有自己的 Agent、提示词、知识结构、连接器、数据、领域评测和用户支持。它可以快速试验，也为自己的系统行为负责，而不必等待通用项目批准每个领域决定。

第三步是证据回流。真实使用中发现的通用缺陷或机制，先去掉材料领域的特殊假设，再补上聚焦测试和兼容说明，作为上游改进提案。上游评审通过后，改进进入共同底座，下一轮又能被不同下游使用。

这不是一条单向生产线，而是一个循环：底座降低领域创新的起点，领域实践帮助底座发现真正的共同能力，证据决定什么值得回流。

ASKS 怎样让这套模式落地？

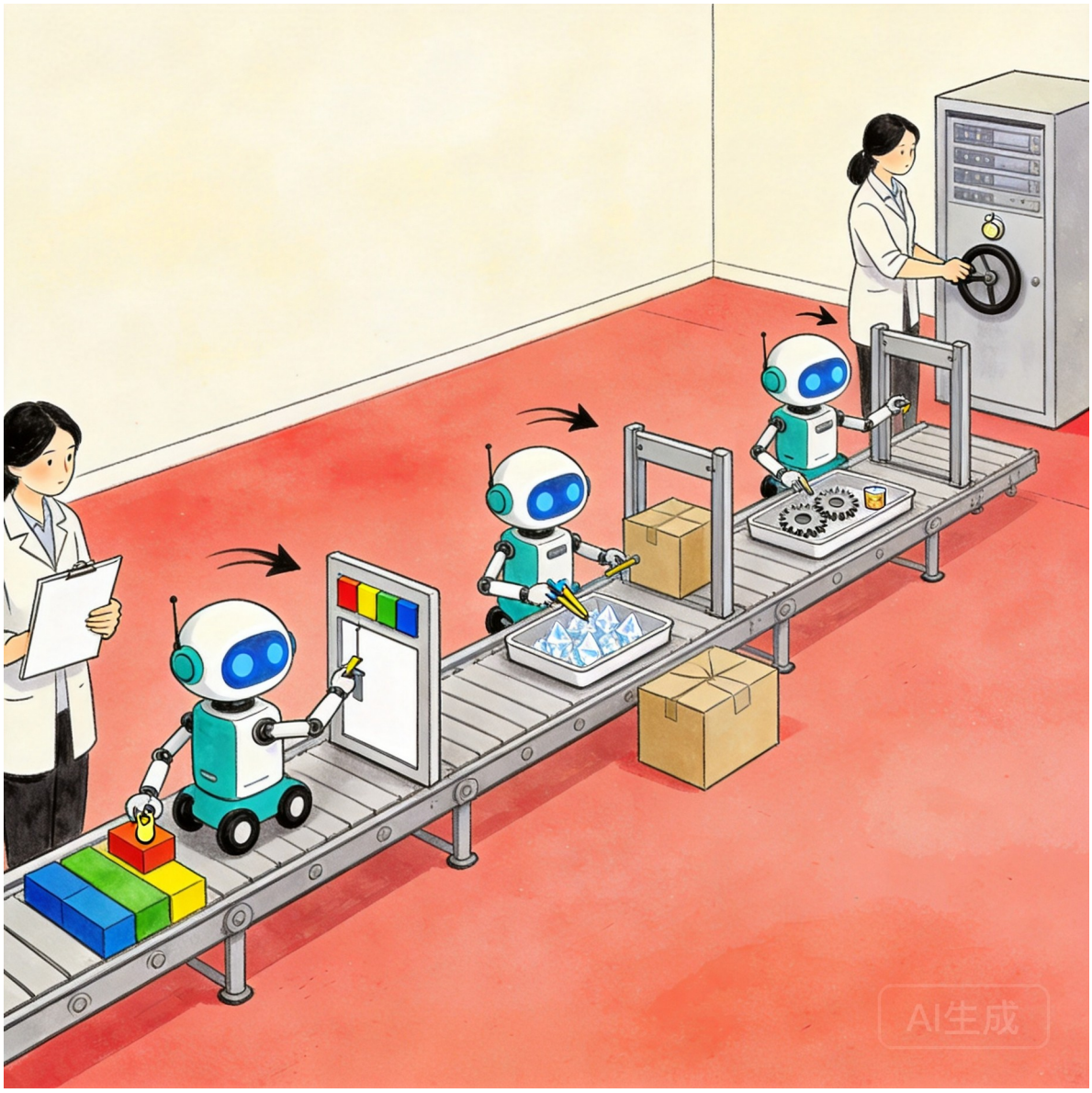
在 ASKS 当前的生态治理中，一个独立团队可以建立 Community Downstream，也就是由自己维护的社区下游。它可以说明自己基于 ASKS 的哪个版本，但领域行为、数据、评测、发布、升级和支持仍由下游团队负责。

为避免一句“我们基于同一个项目”掩盖大量差异，ASKS 提供了社区下游登记模板。材料团队需要记录：使用了哪个确切版本和代码提交，谁负责维护，改了哪些共同部分，运行哪些检查，领域数据有什么边界，哪些变化可能值得提交上游。

如果一个下游希望使用 ASKS-Compatible 身份，就不能只靠自我声明。这个身份要绑定明确的上游基线、协议版本、可复现的检查结果、已知差异、评估范围和日期，并由维护者接受。即便契约检查全部通过，它也只证明约定范围内的兼容；材料 Agent 的专业质量，仍要由单独的领域评测说明。

ASKS 的 GitHub 仓库目前还是从受管权威源生成的公开发布树。因此，外部 Pull Request 是一份可评审的贡献提案。提案被接受后，维护者要把改动带回权威源，保留贡献者署名，完成发布检查，再生成新的公开版本。

它很像出版社收到对公开印刷版的修改建议：编辑不会直接把印刷品当母版涂改，而是把接受的修改放回母版，校对后再出新版。这个流程会随着源权威结构变化而调整，但当前目的很明确：让公开协作与隐私、来源和发布边界同时成立。



工程 Agent 可以连续执行共同维护流程，人负责在授权与责任节点作出决定

具体怎样建立这种协作?

如果材料团队今天就要开始，可以沿着下面的路径操作。

> Agent 可以接手大部分共同维护工作。把目标领域、上游版本、仓库地址和必要权限交给具备终端与文件工具的工程 Agent，它就可以按流程执行建仓、登记、检查、升级和上游提案准备。人负责需要授权并承担责任的决定。

- 第一步，钉住起点。选定一个已经发布的 ASKS 版本和确切 commit，也就是不会随时间移动的代码编号；把 ASKS 仓库保留为上游来源。
- 第二步，建立自己的责任边界。创建由材料团队拥有的仓库，公布维护者、问题反馈渠道、发布办法和升级方式。
- 第三步，填好下游记录。写明系统服务什么领域，改过哪些共同文件，使用哪些协议和检查命令，数据能否公开，适用什么许可证。
- 第四步，分开两种验证。一组检查确认结构、接口和来源记录仍符合约定；另一组用真实材料任务评估 Agent 回答是否专业。前者通过，不能替代后者。
- 第五步，持续跟进上游。在短期分支中试着升级新版本，经评审后再合并，同时记录仍然存在的差异和迁移影响。
- 第六步，给变化分类。材料专属的 Agent、提示词和知识结构留在下游；共同底座的缺陷直接报告；可能通用的新机制进入候选清单。
- 第七步，把候选变成提案。去掉材料领域的默认假设，补充能复现问题和验证修复的测试，说明兼容影响和迁移办法。
- 第八步，按上游规则完成回流。提交聚焦的改进建议，经评审后保留署名进入权威源，运行完整检查，再发布新的公开版本。

给首次建立下游的协作参与者提示词

如果读者希望基于 ASKS 建立自己的领域下游，可以把下面方括号中的内容换成实际信息，再交给 Codex、WorkBuddy 或其他具备工程工具的 Agent：

> 请基于 ASKS 的【版本或 commit】，为【目标领域】建立一个由【维护者】负责的社区下游，目标仓库是【仓库地址】。请先阅读 ASKS 的生态治理、贡献说明和社区下游登记模板，再完成以下工作：固定上游基线；建立独立仓库和维护边界；填写领域范围、共同部分改动、协议检查、领域评

> 测、数据边界与上游候选；分开运行契约检查和领域质量评测；制定后续升级办法；把发现分为领域专属能力、共同底座缺陷和可能通用的机制；为通用候选去除领域假设、补充测试并准备上游提案。每一步都记录修改文件、执行命令、检查结果、已知差异和待确认事项。涉及数据授权、维护责任、ASKS-Compatible 身份申请或最终回灌时，列出需要负责人决定的问题。先输出执行计划和仍需我提供的信息，得到确认后再开始修改仓库。

两边都在发展，Agent 怎样帮助重新融合？

材料系统建立以后，ASKS 主项目会继续更新，材料团队也会增加自己的知识结构、数据连接和专业评测。再次融合时，不能简单地用新版 ASKS 覆盖材料系统，也不能把两边的文件一次性混在一起。

Agent 可以找到双方分开时使用的共同版本，同时整理此后两条路径上的变化：哪些上游改进可以直接吸收，哪些材料能力应继续保留，哪些结构变化需要迁移，哪些本地补丁已经被新版 ASKS 取代。它先在临时升级分支组合这些变化，再分别运行共同规则检查和材料任务评测。

例如，ASKS 新版改变了知识页面结构，而材料团队增加了晶体参数和实验条件。Agent 可以按照新结构迁移这些专业字段；如果材料团队以前自行修复的通用问题已经进入新版 ASKS，它还可以删除重复补丁。材料专家随后确认字段含义、私有数据边界和专业回答没有发生错误，才发布新版材料系统。

> 融合不是让一方覆盖另一方，而是让 Agent 以共同基线为参照，帮助双方判断哪些变化应当吸收、保留、迁移或回流。

给材料团队 Agent 的融合提示词

材料团队可以把版本、仓库和评测信息填入下面的提示词，交给具备代码仓库、终端和文件工具的工程 Agent：

> 请帮助【材料系统名称】从当前基于的 ASKS【旧版本或 commit】升级到 ASKS【新版本或 commit】。材料系统仓库是【仓库地址】，ASKS 上游地址是【上游地址】，材料领域评测是【评测集或执行命令】。请以旧版 ASKS 为共同起点，分别整理上游和材料系统此后的变化，并将差异分为四类：可以直接吸收的上游改进、已被上游取代的本地补丁、需要映射或迁移的冲突、继续保留在材料系统中的领域能力。先建立临时升级分支，输出融合计划、受影响文件、迁移办法、检查命令、回退点和需要材料团队确认的问题。确认后再实施融合，运行 ASKS 的共同规则检查和材料领域评测，并更新上游基线、下游改动记录、已知差异与评估日期。涉及专业字段含义、私有数据授权、删除现有行为、兼容身份或最终发布时，交由负责人确认。

团队不必追求同一种身份。一个社区下游完全可以长期独立维护；只有在证据充分、确有需要时，才申请兼容身份或把通用机制提交上游。

什么值得回到共同底座？

一个实用判断是：先问这项变化离开材料领域后，是否仍然成立。

安全、来源、隐私、数据完整性和事务正确性的缺陷，通常直接具有共同价值。某种新机制如果已经去掉领域假设，有清楚测试，并在另一个独立场景中也有用，就更有理由进入共同底座。相反，材料领域的知识结构、专用提示词、数据库授权和质量标准，继续由最了解它们的人维护，往往更可靠。

第二个独立用例是一份很强的证据，却不是自动进入上游的通行证。最终仍要看正确性、安全、维护成本和共同边界。

结语：共同开发的不必是同一个最终系统

回到那支材料研究团队。它最终做出的系统，可能在界面、知识、Agent 行为和评测上都与 ASKS 不同。协作的价值并不在于消灭这些差异，而在于让它不必重造可靠底座，也不必把真实经验锁在自己的仓库里。

Agent 时代需要的开发协作，可以浓缩为一句话：

> 共享可靠底座，让领域系统独立生长，再用证据把通用进步送回共同工程。

当代码、知识、数据和评测都成为系统的一部分，“大家一起改同一个软件”自然不再是唯一答案。更有生命力的合作，是让通用工程、领域专家和真实用户各自承担责任，又能在经过验证的地方重新汇合。

ASKS arXiv 论文

- Shi-Ju Ran, Kun Zhang, Xi Wu, Liu-Si Yang, and Wen-Jun Li. LLMs Interpret, Embeddings Organize, Graphs Emerge: Agent-Driven Compilation of Scientific Knowledge. arXiv preprint, arXiv:2608.29612 (2026). ASKS 的系统论文，介绍来源保留、持续知识编译和可重放更新。

相关论文推荐

- Carliss Y. Baldwin and Kim B. Clark. The Architecture of Participation: Does Code Architecture Mitigate Free Riding in the Open Source Development Model?. Management Science, Vol. 52, No. 7, pp. 1116-1127 (2006). 从模块化和参与成本出发解释软件架构怎样影响开放协作，适合进一步理解“共享什么、独立什么”。
- Joel West and Siobhán O'Mahony. The Role of Participation Architecture in Growing Sponsored Open Source Communities. Industry and Innovation, Vol. 15, No. 2, pp. 145-168 (2008). 讨论企业支持的开放源码社区怎样设计参与边界、透明度和治理，有助于理解贡献渠道为何也是系统的一部分。
- Maha Shaikh and Ola Henfridsson. Governing Open Source Software Through Coordination Processes. Information and Organization, Vol. 27, No. 2, pp. 116-135 (2017). 从协调过程解释开放源码项目怎样处理分布式贡献与共同方向，适合延伸阅读上下游之间的治理问题。